

Assignment Tasks — Completion Summary

Task 1	Admin / User Login Page Role-based login with JWT API auth. Admin redirects to panel, User to dashboard. Inactive users are blocked with a clear message. CSRF protected.	✓ Done
Task 2	User Registration Page Full registration form: Full Name, Address, Gmail, Contact Number, Date of Birth, Gender, Profile Photo with live preview. Password validation included.	✓ Done
Task 3	Data Table in Dashboard All registered users shown in a searchable, filterable, paginated table. Columns: Name, Email, Phone, Location, Joined Date, Status, Actions.	✓ Done
Task 4	Admin Dashboard — All Users Admin sees all users with stats cards (Total, Active, Inactive, New Today), Chart.js donut chart, recent login activity, and full CRUD actions per user.	✓ Done
Task 5	User Status — Active / Inactive Toggle switch per user. Inactive users cannot log in. AJAX-powered — status updates instantly without page reload. Color-coded badges.	✓ Done

Tech Stack

Backend	Python 3.x, Django 4.2
REST API	Django REST Framework + SimpleJWT (15 endpoints)
Frontend	HTML5, CSS3, JavaScript (Dark themed UI)
Database	SQLite (development ready)
Auth	JWT Bearer Token + Session Authentication
Libraries	Pillow, CORS Headers, Chart.js, Font Awesome

Section 1 — Login Page (Task 1)

Role-based authentication page. Admins are redirected to the Admin Dashboard, regular users to their own dashboard. Inactive users are blocked from logging in.

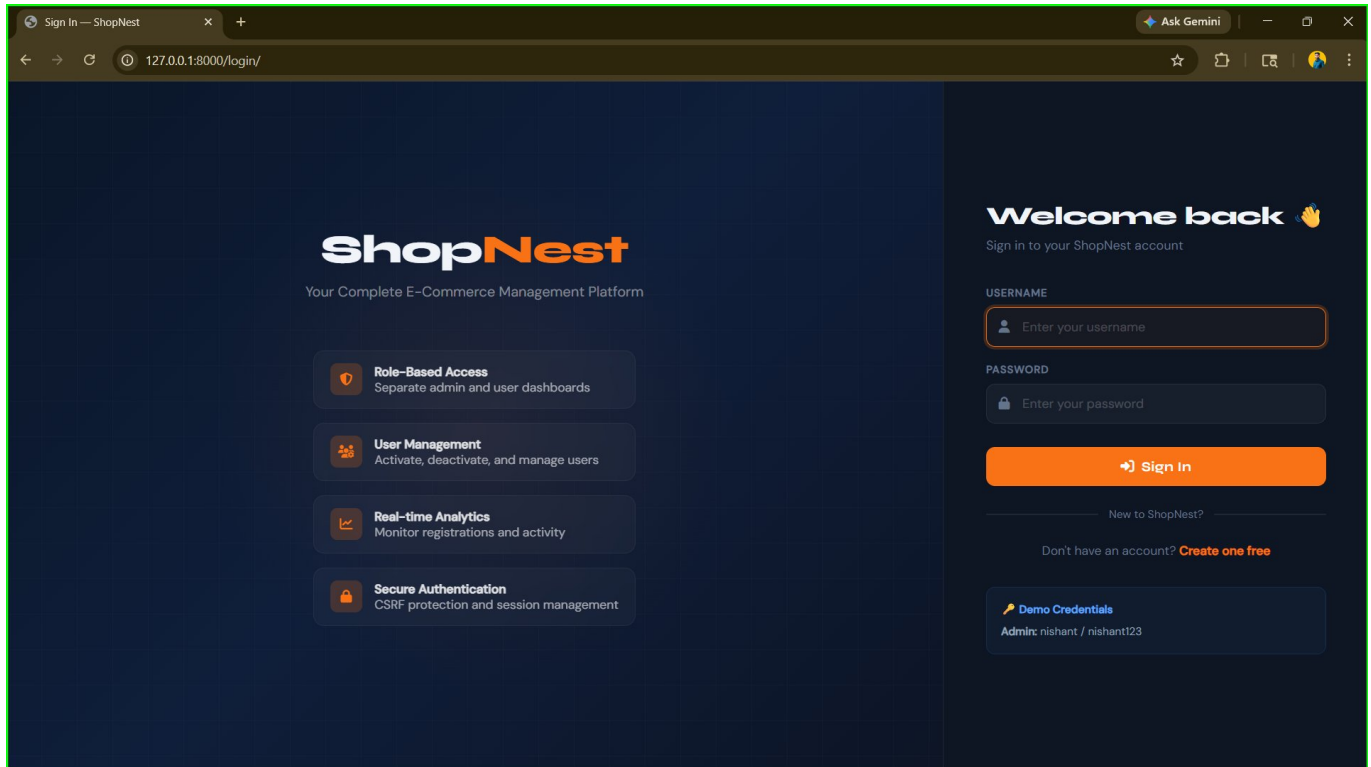


Fig 1.1 — ShopNest Login Page with role-based access and JWT authentication

Section 2 — Admin Dashboard (Tasks 3 & 4)

The admin dashboard shows all registered users in a searchable table with stats cards, chart, recent logins, and per-user action buttons (View, Edit, Delete).

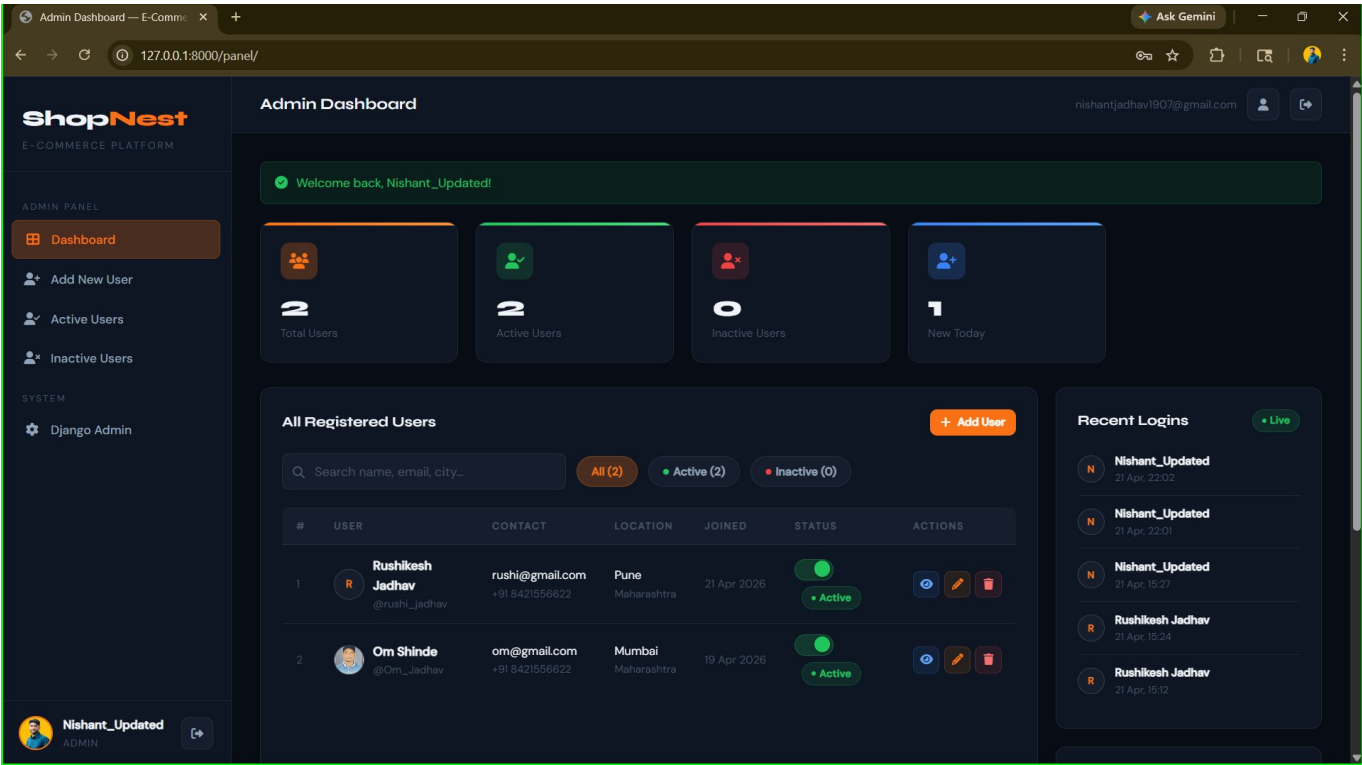


Fig 2.1 — Admin Dashboard with stats cards and full user table

Section 3 — Add New User Form (Task 2)

Admin can create new users directly from the panel. Form includes: Username, Email, Password, Full Name, Phone, DOB, Gender, Profile Photo, and full Address fields.

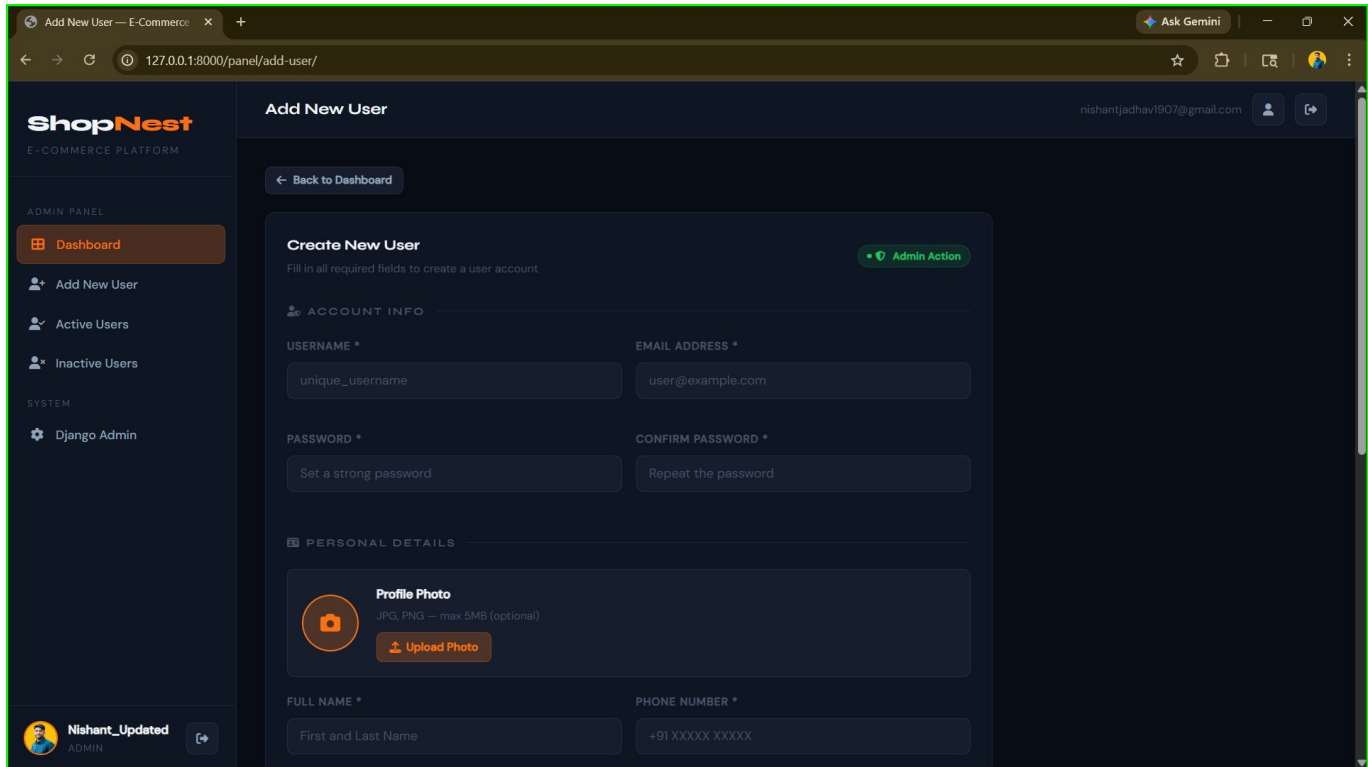


Fig 3.1 — Admin Add New User form with all required fields

Section 4 — Active / Inactive Filter (Task 5)

Admin can filter users by Active or Inactive status. The table updates instantly. Status toggle works via AJAX — no page reload required.

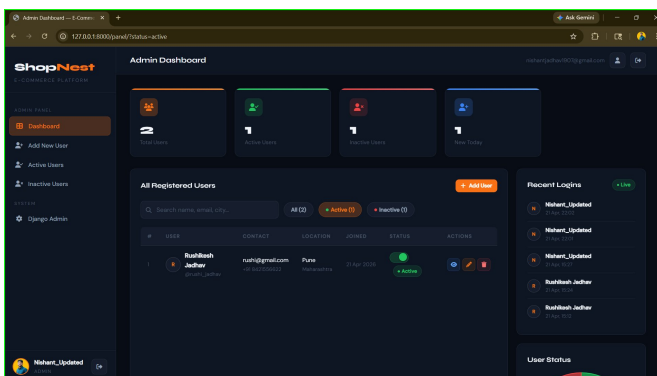


Fig 4.1 — Active users filter (1 active)

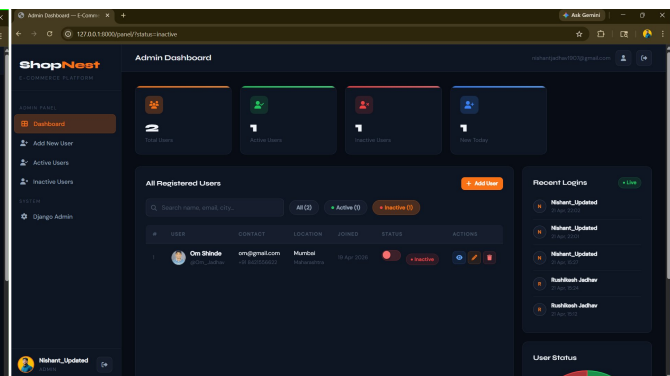


Fig 4.2 — Inactive users filter (1 inactive)

Section 5 — User Dashboard & Profile (Task 1 & 5)

Regular users have their own dashboard showing profile summary, account status (Active/Inactive), and recent login history. Users can also edit their own profile.

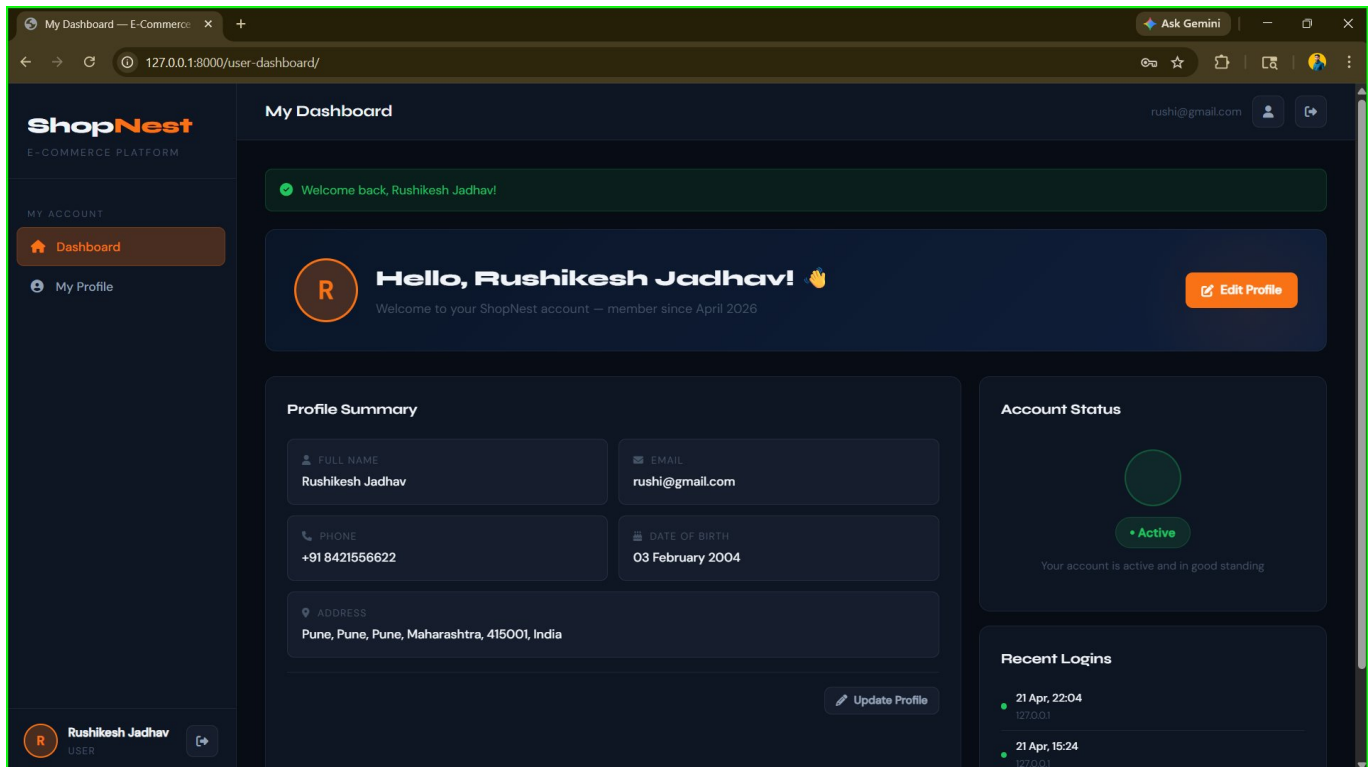


Fig 5.1 — User Dashboard with profile summary and account status

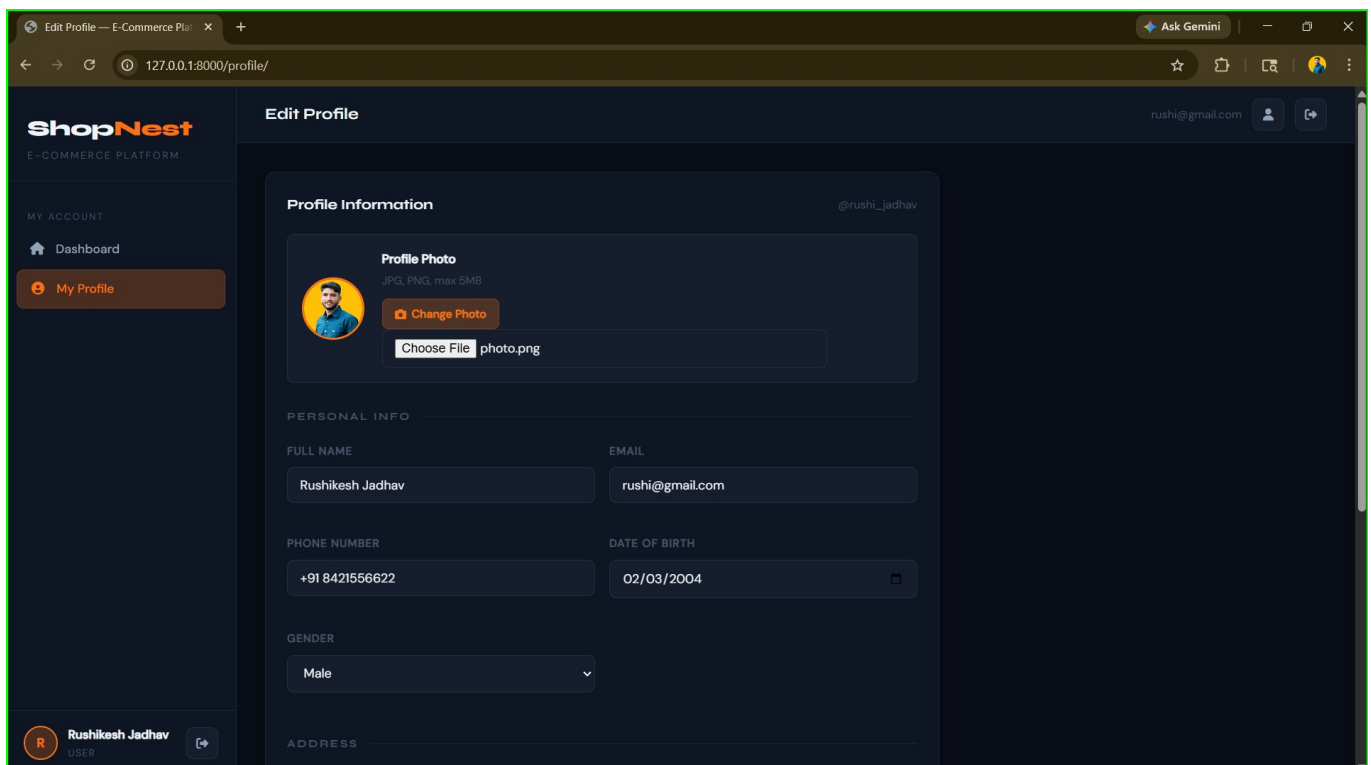


Fig 5.2 — User Edit Profile page with photo upload

Section 6 — REST API — Authentication (JWT Login)

The project includes a full REST API with JWT authentication. Login returns an access token and refresh token used for all secured endpoints.

POST

/api/v1/auth/login/

Login and get JWT access + refresh token

HomeWorkspacesAPI Network

Search PostmanCtrl K

New Environment

HTTP

http://127.0.0.1:8000/api/v1/auth/login/

SaveShare

POST

http://127.0.0.1:8000/api/v1/auth/login/

Send

DocsParamsAuthorizationHeaders (10)BodyScriptsSettings

Cookies

Headers

9 hidden

Key	Value	Description	Bulk Edit	Presets
☒ Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.ey...			
Key	Value	Description		

BodyCookies (1)Headers (10)Test Results

200 OK • 707 ms • 1000 B •

{ JSONPreviewVisualize

```
1 {  
2   "message": "Welcome, nishant!",  
3   "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.  
    eyJ0b2t1b190eXB1oiYmNjZXNzIiwiaXhwIjoibWZlbnR5cCk1MDQsImp0aSI6ImZmNmZkNkZnYyVjYyZTQxYWZhMTUyYwY1YT  
    lmZDczZTBjIiwidXNlcj9pZCI6IjEifQ.MaWbZM6ITihQTIM_5XMhG9xlU12wL8pldWe1JO-MkX4",  
4   "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.  
    eyJ0b2t1b190eXB1oiJoicmVmcmVzaCI6ImV4cCI6MTc3NzM5NDMwNWciaWF0IjoibWZlbnR5cCk1MDQsImp0aSI6ImZmNmZkNkZnYyVjYyZTQxYWZhMTUyYwY1YT  
    cyZmM1MjQwMiIsInVzZXJfawQioiIiwiaXN0IjN1hjcnc1cy8LSj1tUKLJCtmDuVNLMFqWS31CiF58Fo",  
5   "token_type": "Bearer",  
6   "user": {  
7     "id": 1,  
8     "username": "nishant",  
9     "email": "nishant@gmail.com",  
10    "full_name": "nishant",  
11    "is_staff": true,  
12    "is_active": true  
13  }  
14 }
```

Fig 6.1 — POST /api/v1/auth/login/ → 200 OK with JWT token, user info

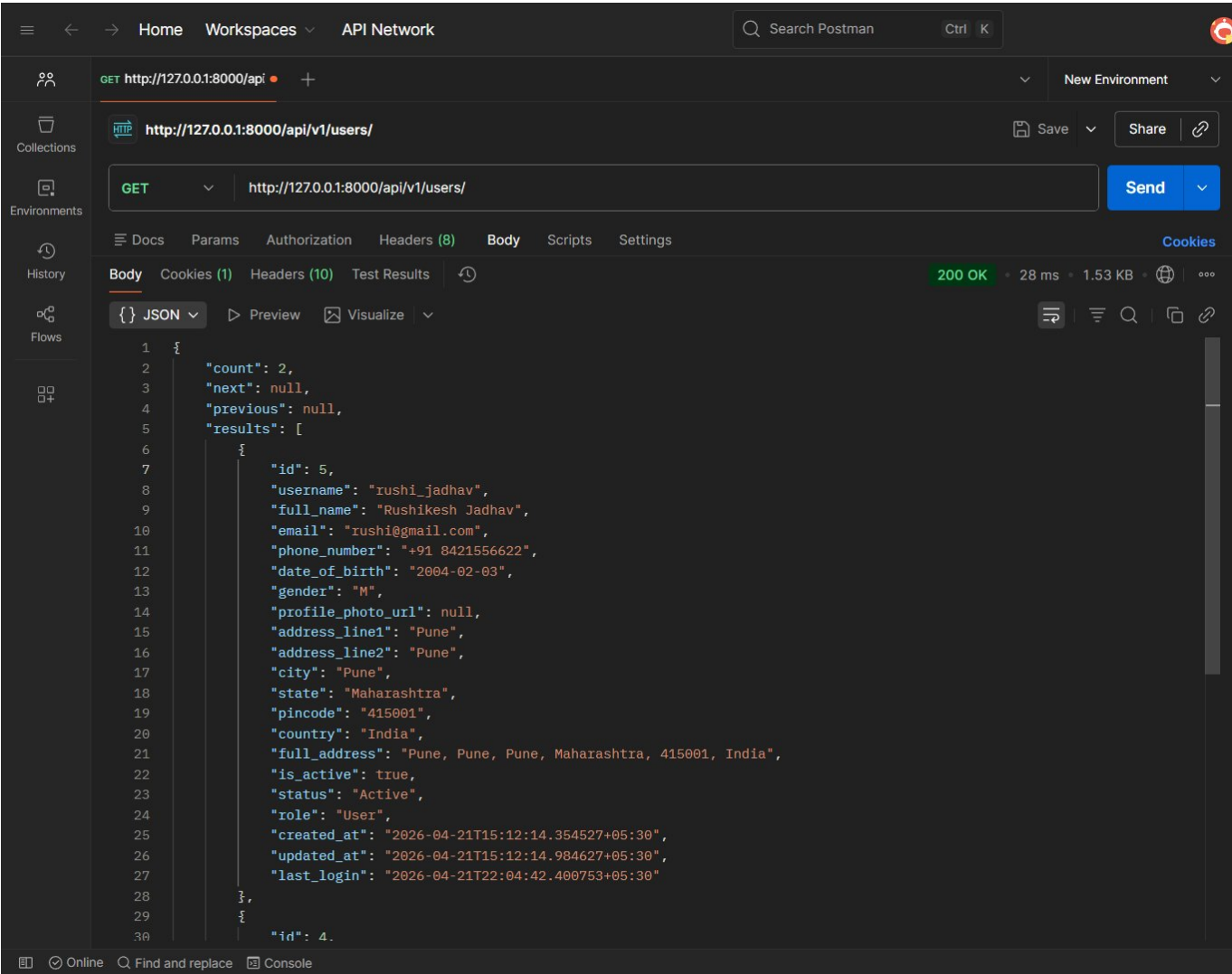
Section 7 — REST API — Get All Users

Admin-only endpoint. Returns paginated list of all registered users with full details including status, role, address, profile photo URL, and login timestamps.

GET

/api/v1/users/

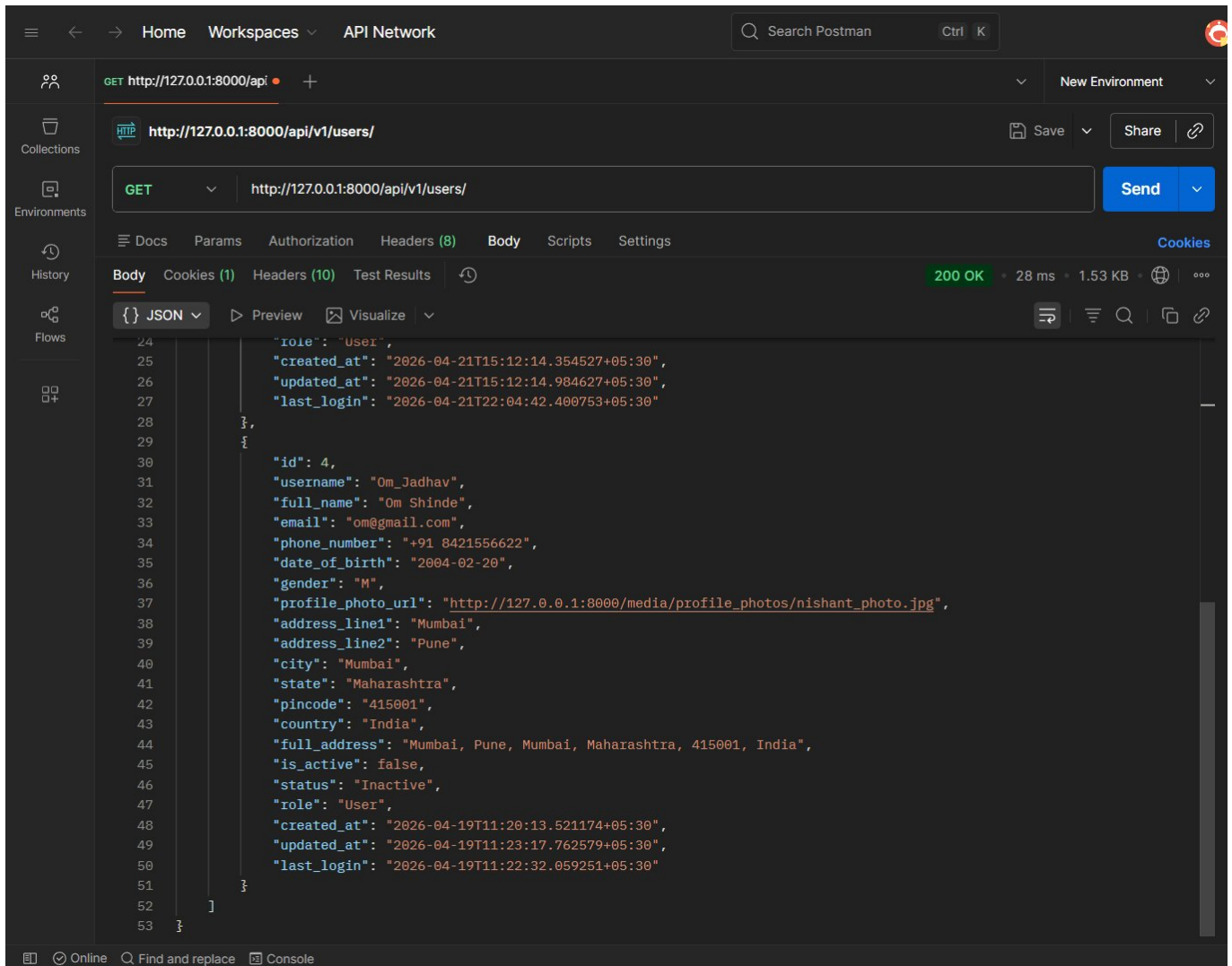
List all users (paginated, admin only)



The screenshot shows the Postman interface with a GET request to `http://127.0.0.1:8000/api/v1/users/` executed successfully. The response is a JSON object with a status of 200 OK, a response time of 28 ms, and a body size of 1.53 KB. The JSON body contains a paginated list of users, with the first user's details visible in the preview.

```
{
  "count": 2,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 5,
      "username": "rush_i_jadhav",
      "full_name": "Rushikesh Jadhav",
      "email": "rush_i@gmail.com",
      "phone_number": "+91 8421556622",
      "date_of_birth": "2004-02-03",
      "gender": "M",
      "profile_photo_url": null,
      "address_line1": "Pune",
      "address_line2": "Pune",
      "city": "Pune",
      "state": "Maharashtra",
      "pincode": "415001",
      "country": "India",
      "full_address": "Pune, Pune, Pune, Maharashtra, 415001, India",
      "is_active": true,
      "status": "Active",
      "role": "User",
      "created_at": "2026-04-21T15:12:14.354527+05:30",
      "updated_at": "2026-04-21T15:12:14.984627+05:30",
      "last_login": "2026-04-21T22:04:42.400753+05:30"
    },
    {
      "id": 4
    }
  ]
}
```

Fig 7.1 — GET /api/v1/users/ → 200 OK — Full user list with all fields



The screenshot shows the Postman interface with a GET request to `http://127.0.0.1:8000/api/v1/users/`. The response is a 200 OK status with a 28 ms response time and 1.53 KB of data. The response body is a JSON array containing two user objects. The second user is inactive and has a profile photo URL.

```
24  {
25    "role": "User",
26    "created_at": "2026-04-21T15:12:14.354527+05:30",
27    "updated_at": "2026-04-21T15:12:14.984627+05:30",
28    "last_login": "2026-04-21T22:04:42.400753+05:30"
29  },
30  {
31    "id": 4,
32    "username": "Om_Jadhav",
33    "full_name": "Om Shinde",
34    "email": "om@gmail.com",
35    "phone_number": "+91 8421556622",
36    "date_of_birth": "2004-02-20",
37    "gender": "M",
38    "profile_photo_url": "http://127.0.0.1:8000/media/profile_photos/nishant_photo.jpg",
39    "address_line1": "Mumbai",
40    "address_line2": "Pune",
41    "city": "Mumbai",
42    "state": "Maharashtra",
43    "pincode": "415001",
44    "country": "India",
45    "full_address": "Mumbai, Pune, Mumbai, Maharashtra, 415001, India",
46    "is_active": false,
47    "status": "Inactive",
48    "role": "User",
49    "created_at": "2026-04-19T11:20:13.521174+05:30",
50    "updated_at": "2026-04-19T11:23:17.762579+05:30",
51    "last_login": "2026-04-19T11:22:32.059251+05:30"
52  }
53 }
```

Fig 7.2 — Second user in results showing inactive status and profile photo URL

Section 8 — REST API — Filter by Status

Users can be filtered by status using query parameters. `?status=active` returns only active users, `?status=inactive` returns only inactive users.

GET	/api/v1/users/?status=active	Filter active users only
GET	/api/v1/users/?status=inactive	Filter inactive users only

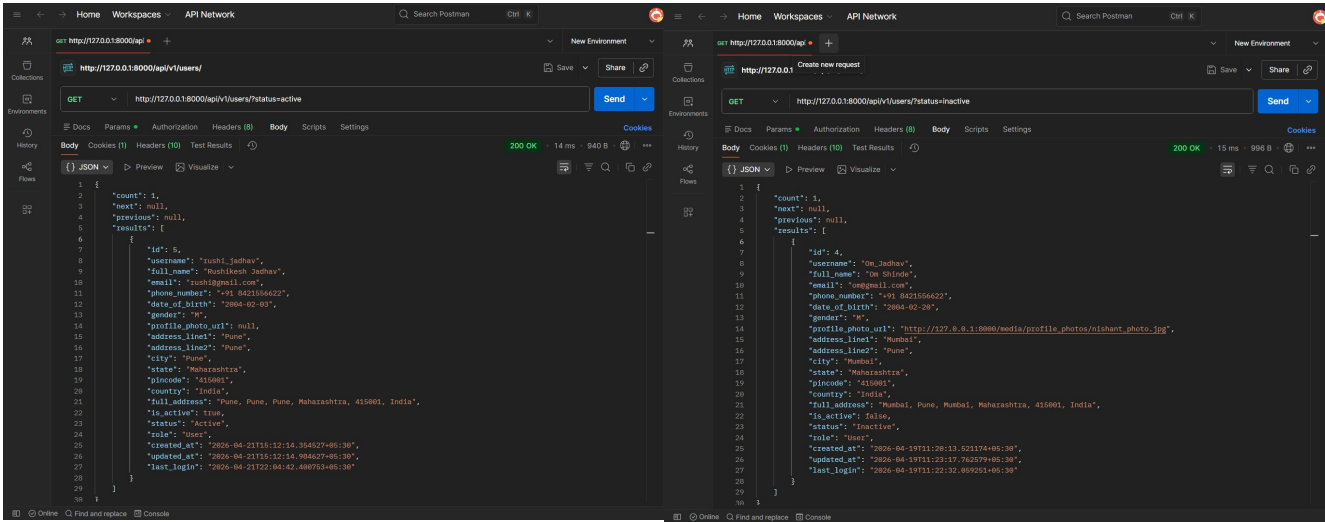


Fig 8.1 — `?status=active` → 1 active user returned

Fig 8.2 — `?status=inactive` → 1 inactive user returned

Section 9 — REST API — Create & Update User

POST /api/v1/users/create/

Create new user (admin only) → 201 Created

PATCH /api/v1/users//update/

Partially update user data → 200 OK

The screenshot shows the Postman interface for a REST client. The request is a POST to `http://127.0.0.1:8000/api/v1/users/create/`. The body is raw JSON, containing user details like username, email, password, full name, phone number, date of birth, gender, and address. The response is a 201 Created status, with a response body showing the created user's details, including an assigned ID and a profile photo URL.

```
1 {
2   "username": "amit_jadhav",
3   "email": "amit@gmail.com",
4   "password": "amit@1234",
5   "password2": "amit@1234",
6   "full_name": "amit jadhav",
7   "phone_number": "+91 99999 88888",
8   "date_of_birth": "1995-06-15",
9   "gender": "M",
10  "address_line1": "123 MG Road",
11  "city": "Pune",
12  "state": "Maharashtra",
13  "pincode": "411001",
14 }
15
16 {
17   "id": 7,
18   "username": "amit_jadhav",
19   "full_name": "amit jadhav",
20   "email": "amit@gmail.com",
21   "phone_number": "+91 99999 88888",
22   "date_of_birth": "1995-06-15",
23   "gender": "M",
24   "gender_display": "Male",
25   "profile_photo_url": null,
26   "address_line1": "123 MG Road",
27   "address_line2": "",
28   "city": "Pune",
29   "state": "Maharashtra",
30   "pincode": "411001"
31 }
```

Fig 9.1 — POST /api/v1/users/create/ → 201 Created with full user detail returned

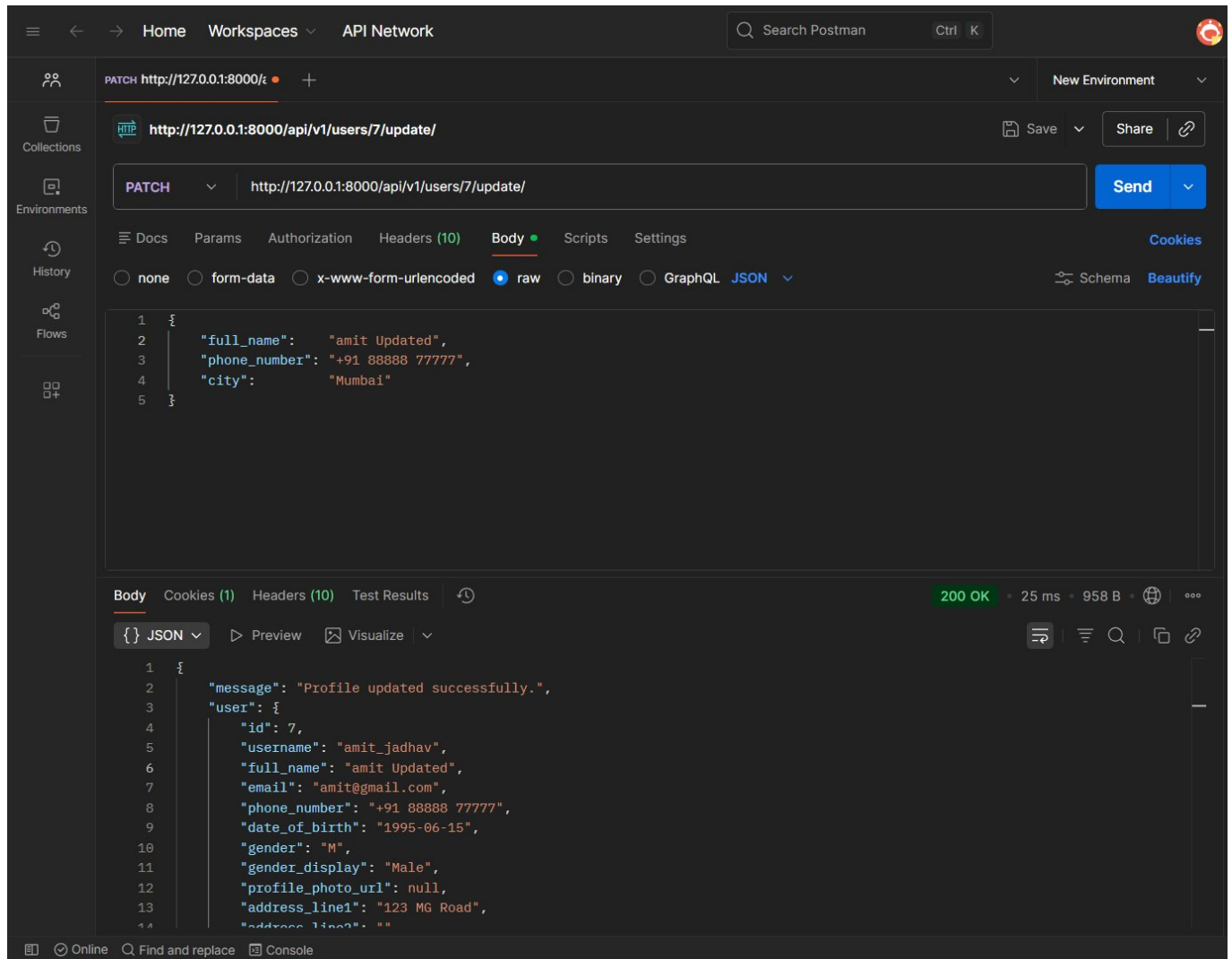


Fig 9.2 — PATCH `/api/v1/users/7/update/` → 200 OK — Profile updated successfully

Section 10 — REST API — Stats & Login Activity

GET	/api/v1/stats/	Dashboard stats — total, active, inactive, new today
GET	/api/v1/login-activity/	Login history — all sessions with IP & timestamp

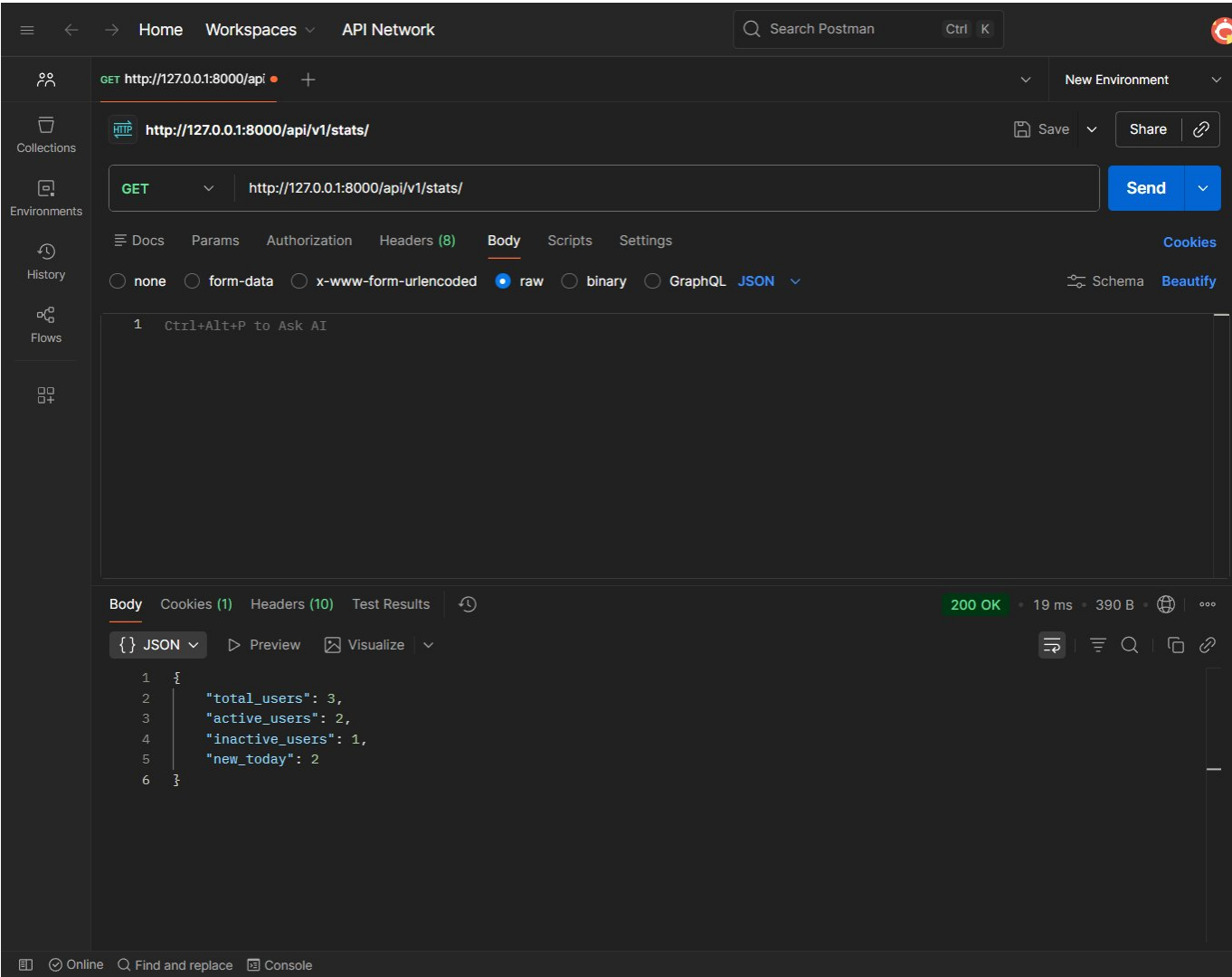
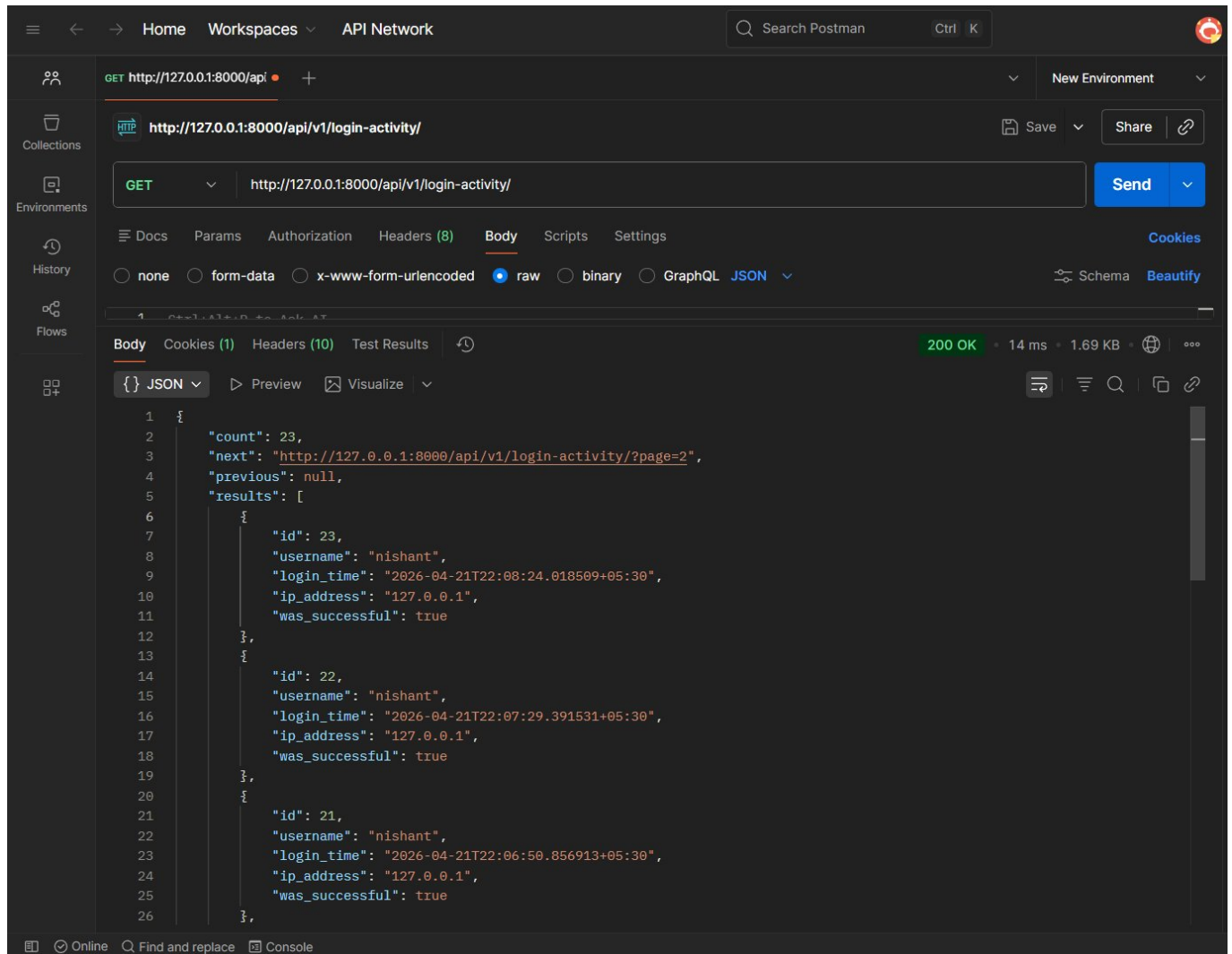


Fig 10.1 — GET /api/v1/stats/ → 200 OK — Dashboard statistics



The screenshot shows the Postman interface with a GET request to `http://127.0.0.1:8000/api/v1/login-activity/` that has been successfully executed. The response status is `200 OK` with a response time of `14 ms` and a size of `1.69 KB`. The response body is displayed in JSON format, showing a paginated list of login records.

```
1 {
2   "count": 23,
3   "next": "http://127.0.0.1:8000/api/v1/login-activity/?page=2",
4   "previous": null,
5   "results": [
6     {
7       "id": 23,
8       "username": "nishant",
9       "login_time": "2026-04-21T22:08:24.018509+05:30",
10      "ip_address": "127.0.0.1",
11      "was_successful": true
12    },
13    {
14      "id": 22,
15      "username": "nishant",
16      "login_time": "2026-04-21T22:07:29.391531+05:30",
17      "ip_address": "127.0.0.1",
18      "was_successful": true
19    },
20    {
21      "id": 21,
22      "username": "nishant",
23      "login_time": "2026-04-21T22:06:50.856913+05:30",
24      "ip_address": "127.0.0.1",
25      "was_successful": true
26    }
27  ]
28 }
```

Fig 10.2 — GET /api/v1/login-activity/ → 200 OK — 23 login records with pagination

Section 11 — Complete API Endpoints Reference

POST	/api/v1/auth/login/	Login and get JWT token
POST	/api/v1/auth/logout/	Logout and blacklist token
POST	/api/v1/auth/token/refresh/	Refresh expired access token
POST	/api/v1/auth/change-password/	Change user password
GET	/api/v1/users/	List all users (admin only)
GET	/api/v1/users/?search=name	Search users by name/email/city
GET	/api/v1/users/?status=active	Filter active users
GET	/api/v1/users/?status=inactive	Filter inactive users
POST	/api/v1/users/create/	Create new user (admin only)
GET	/api/v1/users//	Get single user detail
PATCH	/api/v1/users//update/	Update user profile
DELETE	/api/v1/users//delete/	Delete user permanently
PATCH	/api/v1/users//toggle-status/	Toggle active/inactive status
GET	/api/v1/me/	Get my own profile
PATCH	/api/v1/me/update/	Update my own profile
GET	/api/v1/stats/	Dashboard statistics
GET	/api/v1/login-activity/	Login activity history

All responses return JSON • Auth: Bearer Token • Status: 200/201/400/401/403/404